CHAPTER 10

MUX, DEMUX, Decoder, Encoder, Comparator & Parity

Selectors · distributors · decoders · encoders · comparing numbers · error detection

QUICK TRICK

Five easy memory lines for this chapter: MUX = Many to One. DEMUX = One to Many. Decoder = Few to Many (one line active). Encoder = Many to Few (a code). Parity = count the 1s.

10.1 Multiplexer (MUX)

● **LEVEL 1 · BEGINNER**

What is it? A MUX is a **selector**. It has many data inputs but only **one output**. A few **select lines** decide which input is sent to the output.

Why do we need it? To send many signals through one wire (like many phone calls on one cable), and to build logic functions.

Analogy. A TV remote. Many channels arrive, but you pick **one** to watch.

QUICK TRICK

Size rule: a MUX with **n select lines** has **2^n data inputs**. 2 select lines → 4 inputs (4:1). 3 select lines → 8 inputs (8:1). 4 select lines → 16 inputs.



4:1 MUX behaviour: the select lines S1 S0 choose the input.

S1	S0	Y (output)
0	0	I0
0	1	I1
1	0	I2
1	1	I3

```
2:1 MUX :  Y = S' I0 + S I1
4:1 MUX :  Y = S1' S0' I0 + S1' S0 I1 + S1 S0' I2 + S1 S0 I3
(Each input is ANDed with its select pattern, like a minterm.)
```

QUICK TRICK

Reading the MUX: the select value in binary is the **number of the input** that goes out. S1 S0 = 10 means decimal 2, so **I2** goes to Y.

Implementing a function with a MUX — the main GATE use

Idea. A MUX can produce **any** Boolean function. Think of the select lines as the input variables and the data inputs as the function values.

Method 1 (direct): an n-variable function with a **2^n:1 MUX**. Connect all variables to the select lines. Set $I_k = 1$ if minterm k is in the function, otherwise 0.

Method 2 (smaller MUX): an n-variable function with a **2^{n-1}:1 MUX**. Use the first $n - 1$ variables as select lines. Use the **last variable** (or 0, 1, its complement) at the data inputs.

QUICK TRICK

Pair trick (Method 2). For each select value, look at the **two rows** (last variable = 0, then last variable = 1):
(0, 0) → data input **0** | (1, 1) → **1** | (0, 1) → the **variable itself** (C) | (1, 0) → the **complement** (C').

Example 1: F(A,B,C) = Σm(1, 2, 6, 7) with a 4:1 MUX (S1 = A, S0 = B)

Pair of rows for AB = 00: m0, m1 = (0, 1) → **I0 = C**. AB = 01: m2, m3 = (1, 0) → **I1 = C'**.

AB = 10: m4, m5 = (0, 0) → **I2 = 0**. AB = 11: m6, m7 = (1, 1) → **I3 = 1**.

AB (selects)	C = 0 row	C = 1 row	Pair	Data input
00	m0 = 0	m1 = 1	(0,1)	I0 = **C**
01	m2 = 1	m3 = 0	(1,0)	I1 = **C'**
10	m4 = 0	m5 = 0	(0,0)	I2 = **0**
11	m6 = 1	m7 = 1	(1,1)	I3 = **1**

Example 2: full adder with two 4:1 MUXes (A, B as selects, C as data)

Sum = Σm(1, 2, 4, 7): I0 = C, I1 = C', I2 = C', I3 = C (that is A ⊕ B ⊕ C).

Cout = Σm(3, 5, 6, 7): I0 = 0, I1 = C, I2 = C, I3 = 1 (majority function).

Example 3 (Level 3): F(A,B,C,D) = Σm(0, 1, 3, 4, 8, 9, 15) with an 8:1 MUX (selects A, B, C; data D)

ABC = 000: m0, m1 = (1,1) → **1**. 001: m2, m3 = (0,1) → **D**. 010: m4, m5 = (1,0) → **D'**. 011: m6, m7 = (0,0) → **0**.

ABC = 100: m8, m9 = (1,1) → **1**. 101: m10, m11 = (0,0) → **0**. 110: m12, m13 = (0,0) → **0**. 111: m14, m15 = (0,1) → **D**.

Data inputs: I0 = 1, I1 = D, I2 = D', I3 = 0, I4 = 1, I5 = 0, I6 = 0, I7 = D.

Example 4: XOR using a 2:1 MUX

Select line = A. I0 = B, I1 = B'. When A = 0 output = B, when A = 1 output = B'. That is A ⊕ B. (One MUX and one NOT.)

Cascading (making a big MUX from small MUXes)

QUICK TRICK
Counting trick: number of small k:1 MUXes needed for an N:1 MUX = **(N − 1) / (k − 1)**.
8:1 from 2:1 → 7/1 = **7**. 16:1 from 4:1 → 15/3 = **5** (4 in the first level + 1 in the second). 64:1 from 4:1 → 63/3 = **21**.
The first level uses the **low** select lines. The last level uses the **high** select lines.

GATE POINT
A 2^n:1 MUX can be built from **$2^n - 1$** two-to-one MUXes. 8:1 from 4:1 and 2:1: **two 4:1 + one 2:1**.

10.2 Demultiplexer (DEMUX)

● **LEVEL 1 · BEGINNER**

What is it? The opposite of a MUX. **One input, many outputs.** The select lines decide **which output** receives the data. All other outputs stay 0. **Analogy:** a postman delivering a letter to one of many houses.

S1	S0	Y0	Y1	Y2	Y3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

(The table shows the output when the data input D = 1. If D = 0, all outputs are 0.)

```
Y0 = D S1' S0'     Y1 = D S1' S0     Y2 = D S1 S0'     Y3 = D S1 S0
```

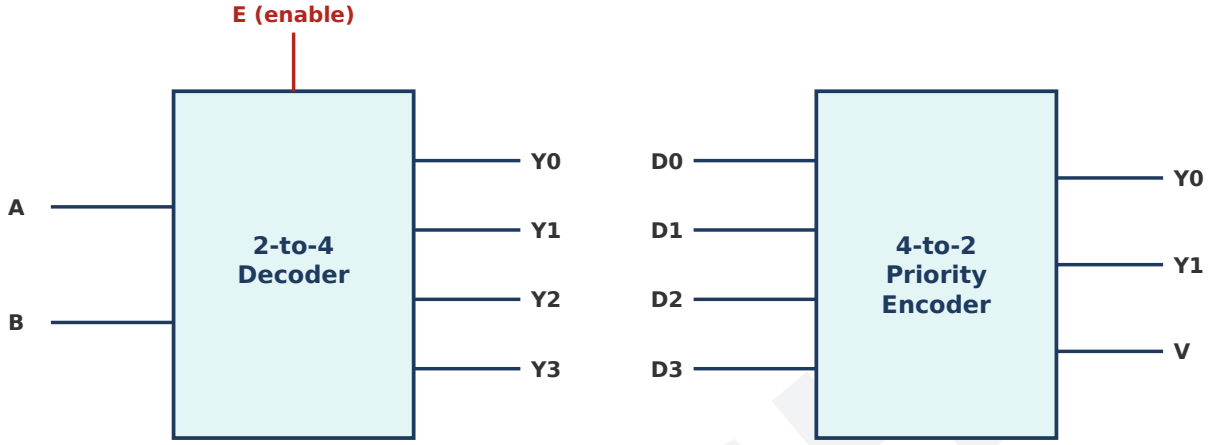
QUICK TRICK
DEMUX with D tied to 1 is a DECODER. And a decoder with an enable input is a DEMUX (the enable is the data input). They are the same circuit used in two ways.

MUX	DEMUX
Many inputs → one output	One input → many outputs
2^n data inputs, n select lines	2^n outputs, n select lines
Data selector	Data distributor
Used to send data, to build functions	Used for routing data, and as a decoder

10.3 Decoder

● LEVEL 1 · BEGINNER

What is it? A decoder takes an **n-bit code** and turns on **exactly one** of its 2^n output lines. **Why?** To select one memory chip, one device, or one row. **Analogy:** typing a room number and only that room's light turns on.



2-to-4 decoder (outputs active-high):

A	B	Y0	Y1	Y2	Y3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

QUICK TRICK

Every decoder output is a minterm. Y0 = A'B', Y1 = A'B, Y2 = AB', Y3 = AB. A 3-to-8 decoder gives m0 to m7. So **a decoder + an OR gate = any function**: OR the minterms you want.

Example: full adder with a 3-to-8 decoder (inputs A, B, Cin)

Sum = Σm(1, 2, 4, 7) → OR gate with inputs Y1, Y2, Y4, Y7.

Cout = Σm(3, 5, 6, 7) → OR gate with inputs Y3, Y5, Y6, Y7.

Total: one 3-to-8 decoder + two 4-input OR gates.

QUICK TRICK

Active-low outputs (like IC 74138): the selected output is 0 and the others are 1. Then use a **NAND** gate instead of OR (NAND of active-low minterms = OR of the minterms). Example: F = Σm(1, 2, 4, 7) → NAND(Y1', Y2', Y4', Y7').

QUICK TRICK

Making big decoders: a 4-to-16 decoder needs **two 3-to-8 decoders** (the 4th input A3 enables one or the other). A 5-to-32 decoder needs **four 3-to-8 decoders and one 2-to-4 decoder** (the 2-to-4 picks which 3-to-8 is on).

10.4 Encoder and priority encoder

● LEVEL 2 · EXAM

What is it? The opposite of a decoder. An encoder has 2^n input lines and gives an **n-bit code** of the line that is active. **Analogy:** a keyboard: pressing a key sends its code.

8-to-3 encoder (only one input is 1 at a time):

```
A2 = D4 + D5 + D6 + D7        (inputs whose number has bit 2 = 1)
A1 = D2 + D3 + D6 + D7        (inputs whose number has bit 1 = 1)
A0 = D1 + D3 + D5 + D7        (inputs whose number has bit 0 = 1)
```

QUICK TRICK

Encoder equation trick: each output bit is the **OR of all inputs whose number has a 1 in that bit**. Look at the binary of the input numbers: D5 = 101 → it feeds A2 and A0.

Problems of a simple encoder: (1) If two inputs are 1 together, the output is wrong. (2) All inputs 0 gives code 000, which looks like D0 is active.

Priority encoder. If several inputs are 1, it outputs the code of the one with the **highest priority** (the highest number). A **Valid** bit V tells us if any input is 1.

D3	D2	D1	D0	Y1	Y0	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

```
Y1 = D3 + D2          Y0 = D3 + D2' D1          V = D3 + D2 + D1 + D0
```

QUICK TRICK

Priority trick: find the **highest 1** from the top (D3 first). Ignore everything below it (those are X). Write the number of that input in binary. Example: D3 D2 D1 D0 = 0110 → highest 1 is D2 → code **10**, V = 1.

Encoder	Decoder
2^n inputs → n outputs	n inputs → 2^n outputs
Output is a binary code	Output is one active line
Example: keyboard, interrupt priority	Example: memory chip select
Large → small	Small → large

10.5 Comparator

● LEVEL 2 · EXAM

What is it? A circuit that compares two numbers and says whether A > B, A = B or A < B. **Why?** Used in sorting, address checking and control.

1-bit comparator:

A	B	A > B	A = B	A < B
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

```
A > B :  A B'          A = B :  A XNOR B  (A B + A' B')          A < B :  A' B
```

Multi-bit comparison. Start from the **leftmost (most significant) bit**. If the bits are different, you already know the answer. If they are equal, look at the next bit.

```
A = B  : every bit equal :   (A3 XNOR B3)(A2 XNOR B2)(A1 XNOR B1)(A0 XNOR B0)
A > B  : A3 B3' + (A3 XNOR B3) A2 B2' + (A3 XNOR B3)(A2 XNOR B2) A1 B1' + ...
```

QUICK TRICK

"Stop at the first difference from the left." A = 1010 and B = 1001: bit 3 equal, bit 2 equal, bit 1: A has 1, B has 0 → **A > B**. You do not even look at bit 0.

GATE POINT

n-bit equality checker = n XNOR gates + one n-input AND gate. **Cascading:** two 4-bit comparators make an 8-bit comparator. The outputs of the lower one go to the cascade inputs of the higher one (the higher comparator uses them only when its own 4 bits are equal).

10.6 Parity

● LEVEL 1 · BEGINNER

What is it? A parity bit is **one extra bit** added to data so that we can **detect errors** during transmission.

Type	Rule
Even parity	Make the **total number of 1s even** (data + parity bit).
Odd parity	Make the **total number of 1s odd**.

QUICK TRICK

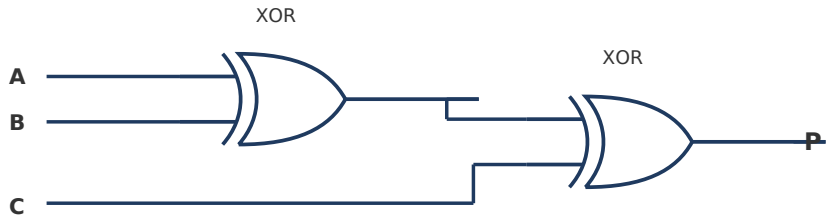
Even parity bit = XOR of all the data bits. (XOR gives 1 when the number of 1s is odd, and the extra 1 makes it even.) **Odd parity bit = the complement of that (XNOR).**

Examples

Data **1011** has three 1s (odd). **Even parity bit = 1** → sent as 1011**1** (four 1s). **Odd parity bit = 0**.

Data **1101001** has four 1s (even). **Even parity bit = 0**. Odd parity bit = 1.

Parity generator (3-bit data, even parity): P = A ⊕ B ⊕ C.



Parity checker. XOR all the received bits (data + parity). For even parity, **0 = no error, 1 = error**.

> **GATE POINT**
> **Limits of parity:** it detects **all odd-number bit errors** (1, 3, 5…). It **cannot detect even-number errors** (2, 4…). It **cannot correct** any error and does not tell which bit is wrong. An n-input parity generator needs **(n − 1) XOR gates**.

Quick-Solve Tricks — Summary

Topic	Trick
MUX size	n select lines → 2^n inputs.
MUX function (Method 2)	Pair of rows: (0,0)→0, (1,1)→1, (0,1)→ variable, (1,0)→ complement.
MUX count	(N − 1) / (k − 1) small k:1 MUXes for an N:1 MUX.
DEMUX	DEMUX with D = 1 is a decoder.
Decoder	Outputs are minterms. Decoder + OR (or NAND for active-low) = any function.
Big decoder	4:16 = two 3:8. 5:32 = four 3:8 + one 2:4.
Encoder	Output bit = OR of inputs whose number has a 1 in that bit.
Priority encoder	Find the highest 1. Ignore the lower inputs.
Comparator	Start at the MSB. Stop at the first difference.
Parity	Even parity bit = XOR of data. Detects odd errors only.

GATE QUICK CHECK — Chapter 10

```
4:1 MUX : Y = S1'S0' I0 + S1'S0 I1 + S1 S0' I2 + S1 S0 I3
n select lines -> 2^n data inputs        Decoder : Yi = minterm i
Priority encoder 4:2 : Y1 = D3 + D2,  Y0 = D3 + D2'D1
1-bit comparator : A>B = AB',  A=B = A XNOR B,  A<B = A'B
Even parity bit = XOR of data bits
Parity generator for n bits : n - 1 XOR gates
```

Traps: select lines vs data inputs · active-low outputs need NAND · a simple encoder fails with two active inputs · parity cannot find even-numbered errors · comparator starts from the MSB.

Practice questions

● **LEVEL 1 · BEGINNER**

Q1. How many select lines does an 8:1 MUX have?

Q2. How many outputs does a 3-to-8 decoder have?

Q3. Which circuit has many inputs and one output: MUX or DEMUX?

● **LEVEL 2 · EXAM**

Q4. In a 4:1 MUX, S1 S0 = 10. Which input goes to the output?

Q5. Implement F(A,B,C) = Σm(0, 3, 5, 6) using a 4:1 MUX with A, B as select lines. Give I0 to I3.

Q6. Find the even parity bit for the data 1101001.

● **LEVEL 3 · GATE**

Q7. How many 4:1 MUXes are needed to build a 64:1 MUX?

Q8. A 4-to-2 priority encoder (D3 highest priority) gets D3 D2 D1 D0 = 0110. Find Y1 Y0 and V.

Q9. How many XNOR gates and AND gates are needed for a 4-bit equality checker?

Q10. Implement F(A,B,C) = Σm(1, 2, 4, 7) using a 3-to-8 decoder with active-low outputs and one more gate.

Q11. Implement a 2-input XNOR using one 4:1 MUX (selects A, B). Give I0 to I3.

Q12. An 8-bit word 10110101 arrives with even parity. Is an error detected? Which errors can parity never detect?

Answers

Q	Solution
1	$2^3 = 8$, so **3** select lines.
2	**8** outputs (Y0 to Y7).
3	**MUX** (Many to One).
4	S1 S0 = 10 = decimal 2 → **I2**.
5	AB = 00: m0 = 1, m1 = 0 → (1,0) → **I0 = C'**. AB = 01: m2 = 0, m3 = 1 → (0,1) → **I1 = C**. AB = 10: m4 = 0, m5 = 1 → **I2 = C**. AB = 11: m6 = 1, m7 = 0 → **I3 = C'**.
6	Count the 1s in 1101001: four (even). Even parity bit = **0**.
7	(64 − 1)/(4 − 1) = 63/3 = **21** MUXes.
8	Highest 1 is D2 → code 10. **Y1 Y0 = 10, V = 1**. (Check: Y1 = D3 + D2 = 1, Y0 = D3 + D2'D1 = 0.)
9	**4 XNOR gates + one 4-input AND gate** (equal only if all four bit pairs are equal).
10	The decoder gives Y1', Y2', Y4', Y7' (active-low). Connect them to a **4-input NAND gate**. The NAND output is F.
11	XNOR = 1 for AB = 00 and 11. **I0 = 1, I1 = 0, I2 = 0, I3 = 1**.
12	Count the 1s in 10110101: 1+0+1+1+0+1+0+1 = 5 (odd). For even parity this is wrong → **error detected**. Parity can never detect **an even number of bit errors** (2, 4…) and cannot correct errors.

Next: Chapter 11 — Sequential Circuits, Latches and Flip-Flops (SR, JK, D, T, race-around, master-slave, setup and hold time).